

[helpers.js](#)

```
import { expect } from "@playwright/test";
import { count } from "node:console";

export const getWeiterBtn = (page) =>
page.locator("#WeiterButton");
export const getPlusBtn = (page) =>
  page.locator('.ui-accordion-content-active
  .btn-number[data-type="plus"]');
export const getMinusBtn = (page) =>
  page.locator('.ui-accordion-content-active
  .btn-number[data-type="minus"]');
export const getCounterInput = (page) =>
  page.locator('.ui-accordion-content-active
  input[type="number"]');
export const getSubmitButton = (page) =>
page.locator("#chooseTerminButton");

const LOCATORS = {
  banner: "#banner",
  kontrastBtn: ".set_contrast_btn",
  spracheBtn: "#easyLanguage",
  langSelectBtn: 'button[aria-label="Sprache wählen"]',
};

// export async function setupPage(page) {
//   await
page.goto("https://termine-reservieren.de/termine/hombu
rg/");
//   await page.getByRole("button", { name:
"Akzeptieren" }).click();
```

```

// }

export async function setupPage(page) {
  await
page.goto("https://termine-reservieren.de/termine/hombu
rg/", {
  waitUntil: "domcontentloaded",
  timeout: 5000,
});
  await page.getByRole("button", { name: "Akzeptieren"
}).click();
}

export async function step_1_SelectDepartment(page,
serviceName) {
  await page.getByRole("button", { name: serviceName
}).click();
}

/**
 * @param 'first' | 'last' position //element's
position
 */
export async function step2_SelectAnliegen(page,
position = "first") {
  const tabs = page.getByRole("tab");
  const plusButtons =
page.locator('.btn-number[data-type="plus"]');

  const targetTab = position === "first" ? tabs.first()
: tabs.last();

```

```

    const targetPlus =
        position === "first" ? plusButtons.first() :
plusButtons.last();

    await targetTab.click();
    await targetPlus.click();
}

export async function closeHinweis(page) {
    await expect(page.getByRole("heading", { name:
"Hinweis" })).toBeVisible();
    await page.locator("#OKButton").click();
}

export async function ensureWeiterButtonState(page,
isEnabled = true) {
    const weiterBtn = page.locator("#WeiterButton");

    if (isEnabled) {
        await expect(weiterBtn).toBeEnabled();
    } else {
        await expect(weiterBtn).toBeDisabled();
    }
}

/**
 * @param 'first' | 'last' position
 */
export async function checkInputValue(page, inputValue,
position = "first") {
    const inputLocator = getCounterInput(page);

```

```
const targetElement =
  position === "first" ? inputLocator.first() :
inputLocator.last();
await expect(targetElement).toHaveValue(inputValue);
}

export async function step3_SelectLocation(page) {
  await page.getByRole("button", { name: "Karte
anzeigen" }).click();
  await
expect(page.locator(".leaflet-container")).toBeVisible(
{
  timeout: 15000,
});
  await page
    .getByRole("button", { name: "" })
    .filter({ has: page.locator(".svg-icon-path") })
    .first()
    .click();
  await page.getByRole("button", { name: "Standort
auswählen" }).click();
}

export async function step4_SelectDate(page) {
  const firstAvailableSlot = page
    .locator(".suggest_btn:not([disabled])")
    .first();
  const noSlotsMessage = page.getByText("Kein freier
Termin verfügbar");
  await firstAvailableSlot.waitFor({ state: "visible"
});
}
```

```

    if (await firstAvailableSlot.count()) {
        await firstAvailableSlot.click();
        await expect(page.getByRole("heading", { name:
"Hinweis" })).toBeVisible();
        await page.getByRole("button", { name: "Ja"
}).click();
    } else if (await noSlotsMessage.isVisible()) {
        console.log("Kein freier Termin verfügbar");
        test.skip(true, "Skipped");
    }
}

export async function step_5_FillForm(page, data) {
    const terminButton = getSubmitButton(page);
    await page.getByTitle("Vorname").fill(data.vorname);
    await page.getByTitle("Nachname").fill(data.nachname);
    await page.locator("#email").fill(data.email);
    await page.locator("#emailwhlgl").fill(data.email);
    await
page.locator("#geburtsdatumYear").fill(data.year);

    if (data.plz) {
        await page.locator("#plz").fill(data.plz);
        await page.locator("#wohnort").fill(data.wohnort);
    }

    await page.getByText("Ich willige ein").click();
}

export async function verifyStep(page, stepNumber) {

```

```

const heading = `Schritt ${stepNumber}`;

await expect(page.getByRole("heading", { name: heading
})).toBeVisible();
}

export async function clickWeiter(page) {
  await page.getByRole("button", { name: "Weiter"
}).click();
}

export async function runNegativeChecks(page) {
  await test.step("Validate all fields from JSON", async
() => {
    for (const field of testData.invalidFields) {
      await validateField(page, field);
    }
  });

  await test.step("Security Check: XSS Injection", async
() => {
    const vornameInput = page.locator("#vorname");
    await
vornameInput.fill('<script>alert("xss")</script>');
    await page.keyboard.press("Tab");
    await
expect(vornameInput).toHaveClass(/wrongvalidate/);
  });

  await test.step("Verify Submit button is disabled",
async () => {

```

```

        await expect(getSubmitButton(page)).toBeDisabled();
    });
}

/**
 * @param {boolean} shouldBeEnabled - true (active),
 * false (disabled)
 */
export async function verifyReservierenButton(page,
    shouldBeEnabled) {
    const terminButton = getSubmitButton(page);
    if (shouldBeEnabled) {
        await expect(terminButton).toBeEnabled();
    } else {
        await expect(terminButton).toBeDisabled();
    }
}

export async function verifyLogo(page) {
    const banner = page.locator(LOCATORS.banner);
    await expect(banner).toBeVisible();
    await expect(banner).toHaveCSS("background-image",
    /url/);
}

export async function verifyKontrastBtnAn(page) {
    const kontrastBtn =
page.locator(LOCATORS.kontrastBtn);
    await expect(kontrastBtn).toBeVisible();
}

```

```
    await
expect(kontrastBtn).toHaveAttribute("aria-label",
"Kontrast an");
}

export async function verifyKontrastBtnAus(page) {
    const kontrastBtn =
page.locator(LOCATORS.kontrastBtn);
    await kontrastBtn.click();
    await
expect(kontrastBtn).toHaveAttribute("aria-label",
"Kontrast aus");
}

export async function verifySprachBtnAn(page) {
    const spracheBtn = page.locator(LOCATORS.spracheBtn);
    const kontrastBtn =
page.locator(LOCATORS.kontrastBtn);
    await expect(spracheBtn).toBeVisible();
    await expect(spracheBtn).toHaveAttribute("aria-label",
"Einfache Sprache an");
    await kontrastBtn.click();
}

export async function verifySprachBtnAus(page) {
    const spracheBtn = page.locator(LOCATORS.spracheBtn);
    const langSelectBtn =
page.locator(LOCATORS.langSelectBtn);
    await spracheBtn.click();
    await expect(spracheBtn).toHaveAttribute(
        "aria-label",
        "Einfache Sprache aus",
    );
};
```

```
    await expect(langSelectBtn).toBeVisible();
    await spracheBtn.click();
}

export async function verifyFooterLinksFunctional(page)
{
    const footerLinks = [
        { id: "#footer_link_help", modalId: "#modal_help" },
        { id: "#footer_link_imprint", modalId:
"#modal_imprint" },
        { id: "#footer_link_privacy", modalId:
"#modal_privacy" },
        { id: "#footer_link_accessibility", modalId:
"#modal_accessibility" },
        { id: "#footer_link_licenses", modalId:
"#modal_licenses" },
    ];

    for (const entry of footerLinks) {
        await page.locator(entry.id).click();
        await
expect(page.locator("#footer_dialog")).toHaveClass(/in/
);
        await
expect(page.locator(entry.modalId)).toBeVisible();
        await page.locator("#close_btn").click();
        await
expect(page.locator("#footer_dialog")).not.toHaveClass(
/in/);
    }
}
```

```
export async function verifyFooterLinksVisible(page) {
  const footerLinks = [
    "#footer_link_help",
    "#footer_link_imprint",
    "#footer_link_privacy",
    "#footer_link_accessibility",
    "#footer_link_licenses",
  ];

  for (const entry of footerLinks) {
    await expect(page.locator(entry)).toBeVisible();
  }
}

/**
 * @param {number} activeStepIndex
 */
export async function verifyStepIndicator(page,
activeStepIndex) {
  const steps = page.locator('ul.nav-tabs
li[role="presentation"]');
  const count = await steps.count();

  for (let i = 0; i < count; i++) {
    const step = steps.nth(i);

    if (i === activeStepIndex - 1) {
      await expect(step).toHaveClass(/active/);
    } else if (i < activeStepIndex - 1) {
      await expect(step).toHaveClass(/done/);
    }
  }
}
```

```

    } else {
        await expect(step).toHaveClass(/disabled/);
    }
}
}

/**
 * @param {number} currentStep
 */
export async function verifyUebersichtData(page,
stepNumber) {
    const rows = page.locator("dl.grid dt");

    const rowsToCheck = stepNumber - 1;

    for (let i = 0; i < rowsToCheck; i++) {
        const dd = rows.nth(i).locator("+ dd");

        await expect(dd).not.toContainText("noch nicht
gesetzt");
    }
}

/**
 * @param {page} page
 * @param {Array<boolean>} expectedStates
 */
export async function verifyUebersichtState(page,
expectedStates) {
    const rows = page.locator("dl.grid dt");

```

```

for (let i = 0; i < expectedStates.length; i++) {
    const dd = rows.nth(i).locator("+ dd");

    if (expectedStates[i] === true) {
        await expect(dd).not.toContainText("noch nicht
gesetzt");
    } else {
        await expect(dd).toContainText("noch nicht
gesetzt");
    }
}

export function getFormattedFutureDate(daysAhead) {
    const date = new Date();
    date.setDate(date.getDate() + daysAhead);

    // Saturday(6) Sunday (0) - shift to friday
    if (date.getDay() === 6) {
        date.setDate(date.getDate() - 1);
    } else if (date.getDay() === 0) {
        date.setDate(date.getDate() - 2);
    }

    return date.toLocaleDateString("de-DE", {
        day: "2-digit",
        month: "2-digit",
        year: "numeric",
    });
}

```

```
export async function validateField(  
  page,  
  fieldId,  
  value,  
  expectedError,  
  errorIndex,  
) {  
  const input = page.locator(fieldId);  
  const errorMsg =  
page.locator(".error-text").nth(errorIndex);  
  
  await input.fill(value);  
  await input.press("Tab");  
  await expect(errorMsg).toHaveText(expectedError ??  
  "");  
}
```

e2e_booking.[spec.js](#)

```
import { test, expect } from "@playwright/test";
import {
  setupPage,
  verifyStep,
  step_1_SelectDepartment,
  step2_SelectAnliegen,
  clickWeiter,
  closeHinweis,
  ensureWeiterButtonState,
  checkInputValue,
  step3_SelectLocation,
  step4_SelectDate,
  step_5_FillForm,
  verifyReservierenButton,
  verifyLogo,
  verifyKontrastBtnAn,
  verifyKontrastBtnAus,
  verifySprachBtnAn,
  verifySprachBtnAus,
  verifyFooterLinksFunctional,
  verifyFooterLinksVisible,
  verifyStepIndicator,
  verifyUebersichtData,
  validateField,
  runNegativeChecks,
  verifyUebersichtState,
  getFormattedFutureDate,
} from "../helpers.js";
import testData from "../testData.json" assert { type:
"json" };

```

```
test.describe("E2E Booking Flows", () => {
  test("TS_01_A - E2E Flow: Bürgeramt", async ({ page })
=> {
    await setupPage(page);
    await verifyStep(page, "1");

    await step_1_SelectDepartment(page, "Bürgeramt");
    await verifyStep(page, "2");

    await step2_SelectAnliegen(page);
    await checkInputValue(page, "1");
    await ensureWeiterButtonState(page, true);
    await clickWeiter(page);
    await closeHinweis(page);
    await verifyStep(page, "3");

    await step3_SelectLocation(page);
    await verifyStep(page, "4");

    await step4_SelectDate(page);
    await verifyStep(page, "5");

    await step_5_FillForm(page,
testData.happyPath_Buergeramt);
    await verifyReservierenButton(page, true);
  });

  test("TS_01_B - E2E Flow: Fahrerlaubnis", async ({
page }) => {
    await setupPage(page);
```

```
    await verifyStep(page, "1");

    await step_1_SelectDepartment(page,
"Fahrerlaubnis");
    await verifyStep(page, "2");

    await step2_SelectAnliegen(page, "last");
    await checkInputValue(page, "1", "last");
    await ensureWeiterButtonState(page, true);
    await clickWeiter(page);
    await closeHinweis(page);
    await verifyStep(page, "3");

    await step3_SelectLocation(page);
    await verifyStep(page, "4");

    await step4_SelectDate(page);
    await verifyStep(page, "5");

    await step_5_FillForm(page,
testData.happyPath_Fahrerlaubnis);
    await verifyReservierenButton(page, true);
  });

  test("TS_04 - Navigation: Backward & Cache Stability",
async ({ page }) => {
    //1. Navigate to Step 5
    await setupPage(page);
    await step_1_SelectDepartment(page, "Bürgeramt");
    await step2_SelectAnliegen(page);
    await clickWeiter(page);
```

```
await closeHinweis(page);
await step3_SelectLocation(page);
await step4_SelectDate(page);
await verifyStepIndicator(page, 5);
await verifyUebersichtData(page, 5);
await verifyUebersichtState(page, [true, true, true,
true]);

// 2. Navigate back to Step 4
await page.locator("#zurueck").first().click();
await verifyStepIndicator(page, 4);
await verifyUebersichtState(page, [true, true, true,
true]);

// 3. Navigate back to Step 3
await page.locator("#zurueck").first().click();
await verifyStepIndicator(page, 3);
await verifyUebersichtState(page, [true, true,
false, true]);

// 4. Navigate back to Step 2
await page.locator("#zurueck").first().click();
await verifyStepIndicator(page, 2);
await verifyUebersichtState(page, [true, false,
false, false]);

// 5. Navigate back to Step 1
await page.locator("#zurueck").first().click();
await verifyStepIndicator(page, 1);
});
});
```



session_timeout.spec.js

```
import { test, expect } from "@playwright/test";
import {
  setupPage,
  verifyStep,
  step_1_SelectDepartment,
  step2_SelectAnliegen,
  clickWeiter,
  closeHinweis,
  ensureWeiterButtonState,
  checkInputValue,
  step3_SelectLocation,
  step4_SelectDate,
  step_5_FillForm,
  verifyReservierenButton,
  verifyLogo,
  verifyKontrastBtnAn,
  verifyKontrastBtnAus,
  verifySprachBtnAn,
  verifySprachBtnAus,
  verifyFooterLinksFunctional,
  verifyFooterLinksVisible,
  verifyStepIndicator,
  verifyUebersichtData,
  validateField,
  runNegativeChecks,
  verifyUebersichtState,
  getFormattedFutureDate,
} from "../helpers.js";
import testData from "../testData.json" assert { type:
"json" };
```

```
test.describe("E2E Booking Flows", () => {
  test("TS_01_A - E2E Flow: Bürgeramt", async ({ page })
=> {
    await setupPage(page);
    await verifyStep(page, "1");

    await step_1_SelectDepartment(page, "Bürgeramt");
    await verifyStep(page, "2");

    await step2_SelectAnliegen(page);
    await checkInputValue(page, "1");
    await ensureWeiterButtonState(page, true);
    await clickWeiter(page);
    await closeHinweis(page);
    await verifyStep(page, "3");

    await step3_SelectLocation(page);
    await verifyStep(page, "4");

    await step4_SelectDate(page);
    await verifyStep(page, "5");

    await step_5_FillForm(page,
testData.happyPath_Buergeramt);
    await verifyReservierenButton(page, true);
  });

  test("TS_01_B - E2E Flow: Fahrerlaubnis", async ({
page }) => {
    await setupPage(page);
```

```
    await verifyStep(page, "1");

    await step_1_SelectDepartment(page,
"Fahrerlaubnis");
    await verifyStep(page, "2");

    await step2_SelectAnliegen(page, "last");
    await checkInputValue(page, "1", "last");
    await ensureWeiterButtonState(page, true);
    await clickWeiter(page);
    await closeHinweis(page);
    await verifyStep(page, "3");

    await step3_SelectLocation(page);
    await verifyStep(page, "4");

    await step4_SelectDate(page);
    await verifyStep(page, "5");

    await step_5_FillForm(page,
testData.happyPath_Fahrerlaubnis);
    await verifyReservierenButton(page, true);
  });

  test("TS_04 - Navigation: Backward & Cache Stability",
async ({ page }) => {
    //1. Navigate to Step 5
    await setupPage(page);
    await step_1_SelectDepartment(page, "Bürgeramt");
    await step2_SelectAnliegen(page);
    await clickWeiter(page);
```

```
    await closeHinweis(page);
    await step3_SelectLocation(page);
    await step4_SelectDate(page);
    await verifyStepIndicator(page, 5);
    await verifyUebersichtData(page, 5);
    await verifyUebersichtState(page, [true, true, true,
true]);

    // 2. Navigate back to Step 4
    await page.locator("#zurueck").first().click();
    await verifyStepIndicator(page, 4);
    await verifyUebersichtState(page, [true, true, true,
true]);

    // 3. Navigate back to Step 3
    await page.locator("#zurueck").first().click();
    await verifyStepIndicator(page, 3);
    await verifyUebersichtState(page, [true, true,
false, true]);

    // 4. Navigate back to Step 2
    await page.locator("#zurueck").first().click();
    await verifyStepIndicator(page, 2);
    await verifyUebersichtState(page, [true, false,
false, false]);

    // 5. Navigate back to Step 1
    await page.locator("#zurueck").first().click();
    await verifyStepIndicator(page, 1);
  });
});
```



system_errors.spec.js

```
import { test, expect } from "@playwright/test";
import {
  setupPage,
  verifyStep,
  step_1_SelectDepartment,
  step2_SelectAnliegen,
  clickWeiter,
  closeHinweis,
  ensureWeiterButtonState,
  checkInputValue,
  step3_SelectLocation,
  step4_SelectDate,
  step_5_FillForm,
  verifyReservierenButton,
  verifyLogo,
  verifyKontrastBtnAn,
  verifyKontrastBtnAus,
  verifySprachBtnAn,
  verifySprachBtnAus,
  verifyFooterLinksFunctional,
  verifyFooterLinksVisible,
  verifyStepIndicator,
  verifyUebersichtData,
  validateField,
  runNegativeChecks,
  verifyUebersichtState,
  getFormattedFutureDate,
} from "../helpers.js";
import testData from "../testData.json" assert { type:
"json" };
```

```
test.describe("System Resilience & Error Handling", ()
=> {
  test('TS_05 - UI: Predictive Slot Display "Nächster
Termin"', async ({
    page,
  }) => {
    await
page.goto("https://termine-reservieren.de/termine/hombu
rg/");
    await page.getByRole("button", { name: "Akzeptieren"
}).click();
    await page.getByRole("button", { name:
"Fahrerlaubnis" }).click();

    await page.getByRole("tab").last().click();
    await
page.locator('.btn-number[data-type="plus"]').last().cl
ick();

    await page.getByRole("button", { name: "Weiter"
}).click();
    await page.locator("#OKButton").click();

    await verifyStep(page, "3");
    const nextTerminValue =
page.locator('dt:has-text("Nächster Termin") + dd');

    // === UI-MOKING ===
    const myDate = getFormattedFutureDate(30);
    const expectedText = `ab ${myDate}, 10:00 Uhr`;
```

```

    // Replace the actual text with our test data right
on the screen

    await nextTerminValue.evaluate((node, date) => {
        node.textContent = date;
    }, expectedText);

    // === CHECKING the result ===

    await expect(page.getByText(myDate)).toBeVisible();
});

test("TS_06 - Filter: No Available Slots Handling",
async ({ page }) => {
    //Blocked by Test Data
});

test("TS_13 - System: API 500 Error Handling", async
({ page }) => {
    await page.route("**/calendar.js", async (route) =>
{
        await route.fulfill({
            status: 500,
            contentType: "application/json",
            body: JSON.stringify({ error: "Internal Server
Error" }),
        });
    });

    await
page.goto("https://termine-reservieren.de/termine/hombu
rg/");

```

```
    await page.getByRole("button", { name: "Akzeptieren"
    }).click();

    await page.getByRole("button", { name: "Bürgeramt"
    }).click();

    await page.getByRole("tab").first().click();
    await
page.locator('.btn-number[data-type="plus"]').first().c
lick();

    await page.getByRole("button", { name: "Weiter"
    }).click();
    await page.locator("#OKButton").click();

    const step4Heading = page.getByRole("heading", {
name: "Schritt 4" });
    await expect(step4Heading).not.toBeVisible();

    const calendarTable =
page.locator("table.sugg_table");
    await expect(calendarTable).not.toBeVisible();

    const errorText = page.getByText(/Fehler/i);
    await expect(errorText).toBeVisible({ timeout: 10000
    });
  });
});
```

system_resilience.[spec.js](#)

```
import { test, expect } from "@playwright/test";
import {
  setupPage,
  verifyStep,
  step_1_SelectDepartment,
  step2_SelectAnliegen,
  clickWeiter,
  closeHinweis,
  ensureWeiterButtonState,
  checkInputValue,
  step3_SelectLocation,
  step4_SelectDate,
  step_5_FillForm,
  verifyReservierenButton,
  verifyLogo,
  verifyKontrastBtnAn,
  verifyKontrastBtnAus,
  verifySprachBtnAn,
  verifySprachBtnAus,
  verifyFooterLinksFunctional,
  verifyFooterLinksVisible,
  verifyStepIndicator,
  verifyUebersichtData,
  validateField,
  runNegativeChecks,
  verifyUebersichtState,
  getFormattedFutureDate,
} from "../helpers.js";
```

```

import testData from "./testData.json" assert { type:
"json" };

test.describe("System Resilience & Error Handling", ()
=> {
  test('TS_05 - UI: Predictive Slot Display "Nächster
Termin"', async ({
    page,
  }) => {
    await
page.goto("https://termine-reservieren.de/termine/hombu
rg/");
    await page.getByRole("button", { name: "Akzeptieren"
}).click();
    await page.getByRole("button", { name:
"Fahrerlaubnis" }).click();

    await page.getByRole("tab").last().click();
    await
page.locator('.btn-number[data-type="plus"]').last().cl
ick();

    await page.getByRole("button", { name: "Weiter"
}).click();
    await page.locator("#OKButton").click();

    await verifyStep(page, "3");
    const nextTerminValue =
page.locator('dt:has-text("Nächster Termin") + dd');

    // === UI-MOKING ===

```

```
const myDate = getFormattedFutureDate(30);
const expectedText = `ab ${myDate}, 10:00 Uhr`;

// Replace the actual text with our test data right
on the screen
await nextTerminValue.evaluate((node, date) => {
    node.textContent = date;
}, expectedText);

// === CHECKING the result ===
await expect(page.getByText(myDate)).toBeVisible();
});

test("TS_06 - Filter: No Available Slots Handling",
async ({ page }) => {
    //Blocked by Test Data
});

test("TS_13 - System: API 500 Error Handling", async
({ page }) => {
    await page.route("**/calendar.js", async (route) =>
{
    await route.fulfill({
        status: 500,
        contentType: "application/json",
        body: JSON.stringify({ error: "Internal Server
Error" }),
    });
});
});
```

```
    await
page.goto("https://termine-reservieren.de/termine/hombu
rg/");

    await page.getByRole("button", { name: "Akzeptieren"
}).click();

    await page.getByRole("button", { name: "Bürgeramt"
}).click();


    await page.getByRole("tab").first().click();
    await
page.locator('.btn-number[data-type="plus"]').first().c
lick();


    await page.getByRole("button", { name: "Weiter"
}).click();
    await page.locator("#OKButton").click();


    const step4Heading = page.getByRole("heading", {
name: "Schritt 4" });
    await expect(step4Heading).not.toBeVisible();


    const calendarTable =
page.locator("table.sugg_table");
    await expect(calendarTable).not.toBeVisible();


    const errorText = page.getByText(/Fehler/i);
    await expect(errorText).toBeVisible({ timeout: 10000
});
});
});
```

testData.json

```
{  
  "happyPath_Buergeramt": {  
    "vorname": "Vorname",  
    "nachname": "nachname",  
    "email": "orname@mail.com",  
    "year": "2001",  
    "plz": "",  
    "wohnort": ""  
  },  
  "happyPath_Fahrerlaubnis": {  
    "vorname": "Vorname",  
    "nachname": "nachname",  
    "email": "orname@mail.com",  
    "year": "2001",  
    "plz": "66322",  
    "wohnort": "Ort"  
  },  
  
  "incompleteData": {  
    "vorname": "Vorname",  
    "nachname": "nachname",  
    "email": "orname@mail.com",  
    "year": "2001",  
    "plz": "66322",  
    "wohnort": ""  
  },  
  
  "positiveTests": {  
    "vorname": [  
      { "val": "Hans-Joachim" },  
    ],  
  },  
}
```

```
    { "val": "O'Connor" },
    { "val": "Müller-Lüdenscheid" },
    {
      "val":
"ÖTJJKBmcKNtHVVAgAÄrÄnVwlHhAjVGadobSbÜÜRÖahVgxRrÄtEfßxd
nyÜBmfXbsaktxpSÜUaSdHÄZTzVRUFßcSpLINTÄakxjGpyqqCpHSNgeQ
kAÖJÜüÄVZwitbhXeÄsdfHGaöbHAuaäWRmbsäÜwnPTzdöaducSzk bXEÖ
yolycJENzeOUcNEVÜbtmNvyZFxJHYEHäzvCW"
    }
  ],
  "nachname": [
    { "val": "Hans-Joachim" },
    { "val": "O'Connor" },
    { "val": "Müller-Lüdenscheid" },
    {
      "val":
"ÖTJJKBmcKNtHVVAgAÄrÄnVwlHhAjVGadobSbÜÜRÖahVgxRrÄtEfßxd
nyÜBmfXbsaktxpSÜUaSdHÄZTzVRUFßcSpLINTÄakxjGpyqqCpHSNgeQ
kAÖJÜüÄVZwitbhXeÄsdfHGaöbHAuaäWRmbsäÜwnPTzdöaducSzk bXEÖ
yolycJENzeOUcNEVÜbtmNvyZFxJHYEHäzvCW"
    }
  ],
  "email": [{ "val": "test@mail.com" }],
  "year": [{ "val": "1896" }, { "val": "2026" }],
  "plz": [{ "val": "1234" }],
  "wohnort": [
    { "val": "St. Ingbert" },
    { "val": "Baden-Baden" },
    { "val": "München" },
    { "val": "D'Amico" }
  ]
]
```

```
},  
"negativeTests": {  
  "vorname": [  
    { "val": "12345" },  
    { "val": "!@#$%" },  
    { "val": "<script>alert(1)</script>" }  
  ],  
  "nachname": [  
    { "val": "12345" },  
    { "val": "!@#$%" },  
    { "val": "<script>alert(1)</script>" }  
  ],  
  "email": [  
    { "val": "testmail.com" },  
    { "val": "test@mail" },  
    { "val": "test @mail.com" },  
    { "val": ">@+/.()" }  
  ],  
  "emailwhlq": [  
    { "val": "testmail.com" },  
    { "val": "test@mail" },  
    { "val": "test @mail.com" }  
  ],  
  "year": [{ "val": "1895" }, { "val": "2027" }],  
  "plz": [{ "val": "123" }, { "val": "123456" }],  
  "wohnort": [  
    { "val": "Ho" },  
    { "val": "12345" },  
    { "val": "Homburg=" },  
    { "val": "Homburg()" }  
  ]  
]
```

```
},  
  
  "errorMessagees": {  
    "vorname": "Bitte geben Sie einen zulässigen  
Vornamen ein.",  
    "nachname": "Bitte geben Sie einen zulässigen  
Nachnamen ein.",  
    "email": "Bitte geben Sie eine zulässige Mailadresse  
ein.",  
    "emailwhlg": "Bitte geben Sie eine zulässige  
Mailadresse ein.",  
    "year": "Bitte geben Sie ein zulässiges Geburtsdatum  
ein",  
    "plz": "Bitte geben Sie eine zulässige Postleitzahl  
ein.",  
    "wohnort": "Bitte geben Sie einen zulässigen Wohnort  
ein."  
  }  
}
```

ui_component.spec.js

```
import { test, expect } from "@playwright/test";

import {
  setupPage,
  verifyStep,
  step_1_SelectDepartment,
  step2_SelectAnliegen,
  clickWeiter,
  closeHinweis,
  ensureWeiterButtonState,
  checkInputValue,
  step3_SelectLocation,
  step4_SelectDate,
  step_5_FillForm,
  verifyReservierenButton,
  verifyLogo,
  verifyKontrastBtnAn,
  verifyKontrastBtnAus,
  verifySprachBtnAn,
  verifySprachBtnAus,
  verifyFooterLinksFunctional,
  verifyFooterLinksVisible,
  verifyStepIndicator,
  verifyUebersichtData,
  getWeiterBtn,
  getPlusBtn,
  getMinusBtn,
  getCounterInput,
  getSubmitButton,
  validateField,
  runNegativeChecks,
```

```
    getFormattedFutureDate,  
  } from "../helpers.js";  
import testData from "../testData.json" assert { type:  
  "json" };  
  
test.describe("UI Components & Consistency", () => {  
  test("TS_02 - UI: Service Counter Limits", async ({  
    page }) => {  
    await setupPage(page);  
    await step_1_SelectDepartment(page,  
      "Fahrerlaubnis");  
  
    const weiterBtn = getWeiterBtn(page);  
    await expect(weiterBtn).toBeDisabled();  
  
    //0 → 1  
    await step2_SelectAnliegen(page, "last");  
    await expect(weiterBtn).toBeEnabled();  
  
    //1 → 0  
    const minusBtn = getMinusBtn(page);  
    await minusBtn.last().click();  
    await expect(weiterBtn).toBeDisabled();  
  
    //0 → 4  
    const plusBtn = getPlusBtn(page);  
    for (let i = 0; i < 4; i++) {  
      await plusBtn.last().click();  
    }  
  
    const input = getCounterInput(page);
```

```
await expect(input.last()).toHaveValue("4");

// Enabled plus buttons count = 0
await expect(plusBtn).toBeDisabled();

// Disabled minus buttons count = 0
const disabledMinusButtons = minusBtn.filter({
  hasNot: page.locator(":disabled"),
});
await expect(disabledMinusButtons).toHaveCount(1);
await expect(weiterBtn).toBeEnabled();

//4 → 3
await minusBtn.last().click();
await expect(input.last()).toHaveValue("3");

//Check every button state
const plusButtons = getPlusBtn(page);
await expect(plusButtons).toBeEnabled();

//a second option for studying
const buttonsCount = await plusButtons.count();
for (let i = 0; i < buttonsCount; i++) {
  await expect(plusButtons.nth(i)).toBeEnabled();
}
await expect(weiterBtn).toBeEnabled();

await expect(plusButtons).toBeEnabled();
});
```

```

test("TS_03 - UI: Filter Interactivity", async ({ page
})) => {
  await setupPage(page);
  await step_1_SelectDepartment(page,
"Fahrerlaubnis");
  await step2_SelectAnliegen(page, "last");
  await clickWeiter(page);
  await closeHinweis(page);
  await step3_SelectLocation(page);
  await verifyStep(page, "4");

  // 4 step with filter
  await page.getByText("Vorschläge filtern").click();
  await
expect(page.getByText("Zeitraum")).toBeVisible();

  const formattedFutureDate =
getFormattedFutureDate(21);
  await
expect(page.locator('input[name="filter_date_to"]')).to
HaveValue(
    formattedFutureDate,
  );

  // --- Filter Application ---
  await page.locator("#suggest_filter_timespan
summary").click();
  await
page.locator('label[for^="filter_timespan-"]').first().
check();

```

```

    await page.locator("#suggest_filter_weekday
summary").click();

    await
page.locator('label[for^="filter_day-"]').first().check
();

    await page.getByRole("button", { name: "Filtern"
}).click();

    // -----

    // Define filter result locators
    const firstAvailableSlot = page
        .locator(".suggest_btn:not([disabled])")
        .first();

    const noSlotsMessage = page.getByText("Kein freier
Termin verfügbar");

    if (await firstAvailableSlot.isVisible()) {
        await firstAvailableSlot.click();
        await expect(
            page.getByRole("heading", { name: "Hinweis" }),
        ).toBeVisible();
        await page.getByRole("button", { name: "Ja"
}).click();

        await verifyStep(page, "5");
    } else {
        await verifyStep(page, "4");
    }
});

test("TS_14 - UI: Global Header & Overview
Consistency", async ({ page }) => {
    await setupPage(page);

```

```
    await verifyLogo(page);
    await verifyKontrastBtnAn(page);
    await verifyKontrastBtnAus(page);
    await verifySprachBtnAn(page);
    await verifySprachBtnAus(page);
    await verifyFooterLinksFunctional(page);

    await step_1_SelectDepartment(page, "Bürgeramt");
    await verifyLogo(page);
    await verifyKontrastBtnAn(page);
    await verifySprachBtnAus(page);
    await verifyFooterLinksVisible(page);
  });

  test("TS_15 - UI: Dynamic Übersicht & Step Indicator",
async ({ page }) => {
    await setupPage(page);
    await verifyStepIndicator(page, 1);

    await step_1_SelectDepartment(page, "Bürgeramt");
    await verifyStepIndicator(page, 2);
    await verifyUebersichtData(page, 2);

    await step2_SelectAnliegen(page);
    await clickWeiter(page);
    await closeHinweis(page);
    await verifyStepIndicator(page, 3);
    await verifyUebersichtData(page, 3);

    await step3_SelectLocation(page);
    await verifyStepIndicator(page, 4);
```

```
    await verifyUebersichtData(page, 4);

    await step4_SelectDate(page);
    await verifyStep(page, 5);
    await verifyStepIndicator(page, 5);
  });
});
```

validation_security.spec.js

```
import { test, expect } from "@playwright/test";
import {
  setupPage,
  verifyStep,
  step_1_SelectDepartment,
  step2_SelectAnliegen,
  clickWeiter,
  closeHinweis,
  ensureWeiterButtonState,
  checkInputValue,
  step3_SelectLocation,
  step4_SelectDate,
  step_5_FillForm,
  verifyReservierenButton,
  verifyLogo,
  verifyKontrastBtnAn,
  verifyKontrastBtnAus,
  verifySprachBtnAn,
  verifySprachBtnAus,
  verifyFooterLinksFunctional,
  verifyFooterLinksVisible,
  verifyStepIndicator,
  verifyUebersichtData,
  validateField,
  runNegativeChecks,
  verifyUebersichtState,
  getFormattedFutureDate,
} from "../helpers.js";
import testData from "../testData.json" assert { type:
"json" };
```

```

test.describe("Security & Validation", () => {
  test("TS_07 - Security: Deep Linking Protection",
    async ({ page }) => {
      await page.goto(
        "https://termine-reservieren.de/termine/homburg/suggest
        ?suggest&mdt=0&cnc-229=4",
        );
      await expect(
        page.getByText("Fehlermeldung: Kein gültiger
        Standort gefunden."),
        ).toBeVisible();
      await expect(page.getByRole("heading", { name:
        "Schritt 4" })).toBeHidden();
    });

  test("TS_08 - Security: Form Validation", async ({
    page }) => {
      await setupPage(page);
      await step_1_SelectDepartment(page,
        "Fahrerlaubnis");
      await step2_SelectAnliegen(page, "last");
      await clickWeiter(page);
      await closeHinweis(page);
      await step3_SelectLocation(page);
      await step4_SelectDate(page);

      // --- Vorname Validation ---
      for (const item of testData.positiveTests.vorname) {

```

```

        await test.step(`Positive Vorname: ${item.val}`,
async () => {
            await validateField(page, "#vorname", item.val,
null, 0);
        });
    }

    for (const item of testData.negativeTests.vorname) {
        await test.step(`Negative Vorname: ${item.val}`,
async () => {
            await validateField(
                page,
                "#vorname",
                item.val,
                testData.errorMessages.vorname,
                0,
            );
        });
    }

    // --- Nachname Validation ---
    for (const item of testData.positiveTests.nachname)
    {
        await test.step(`Positive Nachname: ${item.val}`,
async () => {
            await validateField(page, "#nachname", item.val,
null, 1);
        });
    }

    for (const item of testData.negativeTests.nachname)
    {

```

```

        await test.step(`Negative Nachname: ${item.val}`,
async () => {
    await validateField(
        page,
        "#nachname",
        item.val,
        testData.errorMessages.nachname,
        1,
    );
});
}

// --- E-mail Validation ---
for (const item of testData.negativeTests.email) {
    await test.step(`Negative Email: ${item.val}`,
async () => {
        await validateField(
            page,
            "#email",
            item.val,
            testData.errorMessages.email,
            2,
        );
    });
}

// --- E-Mail - Wiederholung Validation ---
for (const item of testData.negativeTests.emailwhlg)
{
    await test.step(`Negative Email-Wiederholung:
${item.val}`, async () => {

```

```

        await validateField(
            page,
            "#emailwhlg",
            item.val,
            testData.errorMessages.emailwhlg,
            3,
        );
    });
}

// --- Jahr Validation ---
for (const item of testData.positiveTests.year) {
    await test.step(`Positive year: ${item.val}`,
async () => {
        await validateField(page, "#geburtsdatumYear",
item.val, null, 5);
    });
}

for (const item of testData.negativeTests.year) {
    await test.step(`Negative year: ${item.val}`,
async () => {
        await validateField(
            page,
            "#geburtsdatumYear",
            item.val,
            testData.errorMessages.year,
            5,
        );
    });
}

```

```

// --- PLZ Validation ---
for (const item of testData.positiveTests.plz) {
    await test.step(`Positive PLZ: ${item.val}`, async
() => {
        await validateField(page, "#plz", item.val,
null, 8);
    });
}
for (const item of testData.negativeTests.plz) {
    await test.step(`Negative PLZ: ${item.val}`, async
() => {
        await validateField(
            page,
            "#plz",
            item.val,
            testData.errorMessages.plz,
            8,
        );
    });
}

// --- Wohnort Validation ---
for (const item of testData.positiveTests.wohnort) {
    await test.step(`Positive wohnort: ${item.val}`,
async () => {
        await validateField(page, "#wohnort", item.val,
null, 9);
    });
}
for (const item of testData.negativeTests.wohnort) {

```

```

        await test.step(`Negative wohnort: ${item.val}`,
async () => {
    await validateField(
        page,
        "#wohnort",
        item.val,
        testData.errorMessages.wohnort,
        9,
    );
});
}

// --- Should show error when emails do not match
---

await page.fill("#email", "test@test.de");
await page.fill("#emailwhlg", "wrong@test.de");
await page.press("#emailwhlg", "Tab");

const errorMsg = page.locator(".error-text").nth(3);
await expect(errorMsg).toHaveText(
    "Die Mailadressen stimmen nicht überein.",
);
});

test("TS_16 - System: Email Confirmation Paste
Restriction", async ({
    page,
}) => {
    await setupPage(page);

```

```
    await step_1_SelectDepartment(page,
"Fahrerlaubnis");

    await step2_SelectAnliegen(page, "last");

    await clickWeiter(page);

    await closeHinweis(page);

    await step3_SelectLocation(page);

    await step4_SelectDate(page);


    const emailConfirmField =
page.locator("#emailwhlg");

    await emailConfirmField.click();


    // 3. Emulate a paste event using page.evaluate:
    // We create a 'paste' event and inject data into
its clipboardData.

    await page.evaluate(() => {
        const field =
document.querySelector("#emailwhlg");

        const dataTransfer = new DataTransfer();

        dataTransfer.setData("text", "test@example.com");

        const event = new ClipboardEvent("paste", {
            clipboardData: dataTransfer,
            bubbles: true,

        });

        field.dispatchEvent(event);

    });

    const value = await emailConfirmField.inputValue();

    expect(value).toBe("");

});
```

```
test("TS_17 - System: Error Message on Email
Mismatch", async ({ page }) => {
  // 1. Setup (Deine Schritte)
  await setupPage(page);
  await step_1_SelectDepartment(page,
"Fahrerlaubnis");
  await step2_SelectAnliegen(page, "last");
  await clickWeiter(page);
  await closeHinweis(page);
  await step3_SelectLocation(page);
  await step4_SelectDate(page);

  // 2. Felder lokalisieren
  const email = page.locator("#email");
  const emailConfirm = page.locator("#emailwhlg");
  const errorElement =
page.locator(".error-text").nth(3); // Index, wo die
Fehlermeldung erscheint

  // 3. Aktion: Unterschiedliche E-Mails eingeben
  await email.fill("test@example.com");
  await emailConfirm.fill("wrong@example.com");
  await emailConfirm.press("Tab"); // Trigger für die
Validierung

  // 4. Erwartung
  await expect(errorElement).toHaveText(
    "Die Mailadressen stimmen nicht überein.",
  );
});
```

```
test("TS_18 - System: Validation on Incomplete Form",
async ({ page }) => {
    await setupPage(page);
    await step_1_SelectDepartment(page,
"Fahrerlaubnis");
    await step2_SelectAnliegen(page, "last");
    await clickWeiter(page);
    await closeHinweis(page);
    await step3_SelectLocation(page);
    await step4_SelectDate(page);
    await step_5_FillForm(page,
testData.incompleteData);
    await verifyReservierenButton(page, false);
});
});
```